Software Design Document

for

BrowseBlock

AI/ML-Powered Real-Time Browser Security and Threat Prevention Platform

Prepared by,

Deon George (SJC23CS086)
Amal Kurian Royce (SJC23CS037)
Maria Royce (SJC23CS159)
Melbin K Thomas (SJC23CS164)

Organization: St. Joseph's College of Engineering and Technology, Palai

Date: 26/10/26

Contents

1 System Module Description **2**

1.1 Common Hardware, Software and Database Baseline 2
1.2 Architectural Design 3
1.3 Module 1: Authentication and Access Control Module 3
1.4 Module 2: Browser Extension and Event Monitoring Module 3
1.5 Module 3: URL and QR Phishing Detection Module 3
1.6 Module 4: Webpage and Script Threat Detection Module 4
1.7 Module 5: Malicious File Download Analysis Module 4
1.8 Module 6: Email Phishing Analysis Module 4
1.9 Module 7: Risk Decision and Protective Action Module 5
1.10 Module 8: FastAPI Orchestration and API Module 5
1.11 Module 9: React Web Dashboard Module 5
1.12 Module 10: Detection History, Evidence and Reporting Module 5
1.13 Module 11: Model, Configuration and Monitoring Module 5

2 Diagrams **6**

2.1 Architecture Diagram 6
2.2 Use Case Diagram 6
2.3 Activity Diagram 7
2.4 ER Diagram 8

3 Design Notes and Traceability **8**

1 System Module Description

1.1 Common Hardware, Software and Database Baseline

BrowseBlock is a browser security platform consisting of a Manifest V3 browser extension, a React web dashboard, a Python FastAPI backend, machine-learning and AI analysis services, and MongoDB application storage. The extension is the lightweight, event-driven protection layer. The dashboard provides manual scans, history, reports, analytics, and explanations. Computationally intensive or centralized analysis is coordinated by the backend.

Hardware Baseline

- **Client device:** Desktop or laptop capable of running Google Chrome or a compatible Chromium-based browser.
- **Processor and memory:** A modern multi-core processor and at least 8 GB RAM are recommended for the browser, dashboard, and local development.
- **Storage:** Sufficient storage for source code, models, logs, scan metadata, and controlled temporary file-analysis data.
- **Network:** Stable connectivity for the extension, dashboard, FastAPI, Firebase Authentication, MongoDB, and enabled AI or threat-intelligence services.
- **Optional controlled download environment:** A native helper or download gateway may be used when pre-use file quarantine is required.

Software Baseline

- **Browser:** Google Chrome and compatible Chromium browsers supporting the required extension APIs and Manifest V3.
- **Dashboard:** React, Tailwind CSS, HTML5, CSS3, JavaScript, and Axios or an equivalent HTTP client.
- **Backend:** Python and FastAPI for REST endpoints, validation, authorization, scan coordination, and error handling.
- **Security and AI:** XGBoost for URL and applicable file classification, Random Forest for initial email screening, SHAP where appropriate, and a configured RAG/LLM pipeline for detailed email analysis.
- **Analysis libraries:** QR-decoding, file-format parsers, static-analysis libraries, Playwright-based inspection, and standard Python data-processing tools.
- **Identity and database:** Firebase Authentication with server-side Firebase ID-token verification; MongoDB accessed only by authorized backend services.
- **Development tools:** Visual Studio Code, Git, GitHub, Postman, and an isolated Python environment.

Database Baseline

MongoDB stores user profile metadata, scans, detection results, threat evidence, protective actions, reports, and relevant system records. Firebase Authentication manages identities and credentials; passwords and authentication secrets are not stored in MongoDB. The logical data model is represented by the supplied ER diagram. Each protected record is associated with its authenticated user or an explicitly authorized role. Sensitive URLs, webpage contents, email contents, file contents, and tokens are minimized and retained only according to the configured privacy policy.

1.2 Architectural Design

The system follows a layered client–service architecture. Browser events are observed by the extension, normalized into scan requests, and evaluated locally where low latency is important. The FastAPI service performs authentication, authorization, orchestration, model invocation, and persistence. The dashboard uses the same protected API for user-initiated analysis and historical views. External reputation, AI, and threat-intelligence services are optional dependencies; their unavailability is reported as incomplete analysis rather than as a safe result.

1.3 Module 1: Authentication and Access Control Module

This module integrates Firebase Authentication for registration, sign-in, sign-out, and identity management. The frontend obtains an ID token and includes it with protected API requests. FastAPI verifies the token server-side before processing a request, derives the authenticated user identity, and applies ownership or role-based authorization. The module prevents direct database access from the extension or dashboard and returns explicit authentication or authorization errors.

1.4 Module 2: Browser Extension and Event Monitoring Module

This module implements the Manifest V3 extension and its event-driven protection logic. It observes supported navigation, page content, script, form, frame, notification, extension, and download events without collecting more information than required for a decision. It normalizes observable evidence and dispatches it to the relevant detector. The module distinguishes monitoring from enforcement: the UI reports an activity as blocked only after the browser API confirms the protective action.

1.5 Module 3: URL and QR Phishing Detection Module

This module normalizes URLs and extracts features such as URL length, IP-address usage, domain age, subdomains, suspicious tokens, and brand-impersonation indicators. An XGBoost classifier, configured rules, and available reputation data produce a risk assessment. QR inputs are decoded where supported and the embedded URL is passed through the same URL pipeline. The module returns a risk level, confidence, indicators, and analysis-status information for the warning and reporting layers.

1.6 Module 4: Webpage and Script Threat Detection Module

This module groups the event detectors that operate on live pages:

- **Cross-site scripting:** detects reflected, stored, and DOM-based indicators using observable content and behavior, while avoiding a claim that server-side payloads were removed.
- **Credential harvesting:** examines external script domains, deceptive login interfaces, overlays, pop-ups, and requests for sensitive information without collecting actual passwords.
- **HTML/JavaScript redirects:** tracks observable HTTP, JavaScript, and meta-refresh navigation and evaluates the destination and redirect chain.
- **Clickjacking:** checks framing and overlay indicators and warns or restricts suspicious interaction where browser capabilities permit.
- **Cryptojacking:** evaluates suspicious resource and computation indicators and reports the evidence supporting the risk decision.
- **Notification scams and extension abuse:** inspects observable notification requests, deceptive permission prompts, and suspicious extension activity using supported browser APIs.

Each detector emits structured evidence, a risk assessment, and a proposed protective action. Incomplete browser visibility is reported explicitly.

1.7 Module 5: Malicious File Download Analysis Module

This module monitors supported downloads and identifies *.exe*, *.zip*, *.pdf*, *.doc*, and *.docx* files using names, MIME types, and content-based verification where bytes are available. It extracts format-specific features: PE headers and imports, PDF JavaScript and embedded objects, document macros/OLE objects and relationships, and archive contents, paths, nesting, and compression characteristics. SHA-256 hashes and configured reputation results are combined with deterministic rules and an XGBoost classifier.

Untrusted files are never executed for analysis. File size, time, archive-depth, decompression, entry-count, processing-time, and resource limits are enforced. Malformed files, unsupported formats, parser errors, and timeouts are held or warned according to policy and are never silently classified as safe. A native helper or controlled gateway may quarantine a file before use where browser APIs alone cannot enforce pre-use protection.

1.8 Module 6: Email Phishing Analysis Module

The email module provides two analysis stages. The extension performs lightweight screening with a Random Forest classifier and available SPF, DKIM, and DMARC signals. The dashboard can request deeper analysis using retrieval-augmented generation and configured language-model services. This workflow identifies social engineering, impersonation, urgency,

credential-harvesting intent, and embedded URLs. Extracted URLs are passed to the URL phishing module so that email and URL evidence can be combined in one decision.

1.9 Module 7: Risk Decision and Protective Action Module

This module combines detector outputs, model confidence, reputation information, and analysis completeness into the normalized levels *Safe*, *Suspicious*, and *High-risk*. It selects a warning, restriction, block, cancel, quarantine, release, or allow action according to policy and available browser or helper capabilities. User actions are recorded, and a failed or unavailable enforcement mechanism is shown as such. The module prevents a model prediction alone from being treated as proof of a successful block.

1.10 Module 8: FastAPI Orchestration and API Module

The backend exposes protected REST endpoints for extension events, URL/QR/email/ file scans, reports, history, and analytics. It validates request schemas, verifies Firebase tokens, enforces authorization, coordinates detector services, applies resource limits, and returns consistent result objects. It handles asynchronous I/O where appropriate and surfaces validation, timeout, unsupported input, external-service, and analysis errors instead of returning success-shaped fallbacks.

1.11 Module 9: React Web Dashboard Module

The dashboard provides sign-in, overview, manual scan forms, scan-result views, threat evidence, detection history, reports, and analytics. It supports URL, QR-code, email, and supported-file workflows. Results show the risk level, confidence, indicators, analysis status, and protective action. The interface is designed for non-technical users while retaining detailed evidence for guides, developers, testers, and security analysts.

1.12 Module 10: Detection History, Evidence and Reporting Module

This module persists scans, detection results, evidence, protective actions, and threat reports in MongoDB and retrieves only records authorized for the current user or role. Reports summarize the threat category, risk, confidence, supporting indicators, analysis limitations, and action taken. Retention and privacy policies control whether raw URLs, page content, email content, or file data are retained. The dashboard can present historical results and analytics without exposing authentication secrets or unrelated users' records.

1.13 Module 11: Model, Configuration and Monitoring Module

This module manages versioned detector configuration, model metadata, supported file types, thresholds, browser-capability assumptions, and operational health information. It supports validation data and model evaluation, especially before high-impact file blocking. It records incomplete analyses and integration limitations so that coverage and reliability can be monitored without claiming that every browser attack is preventable.

2 Diagrams

2.1 Architecture Diagram

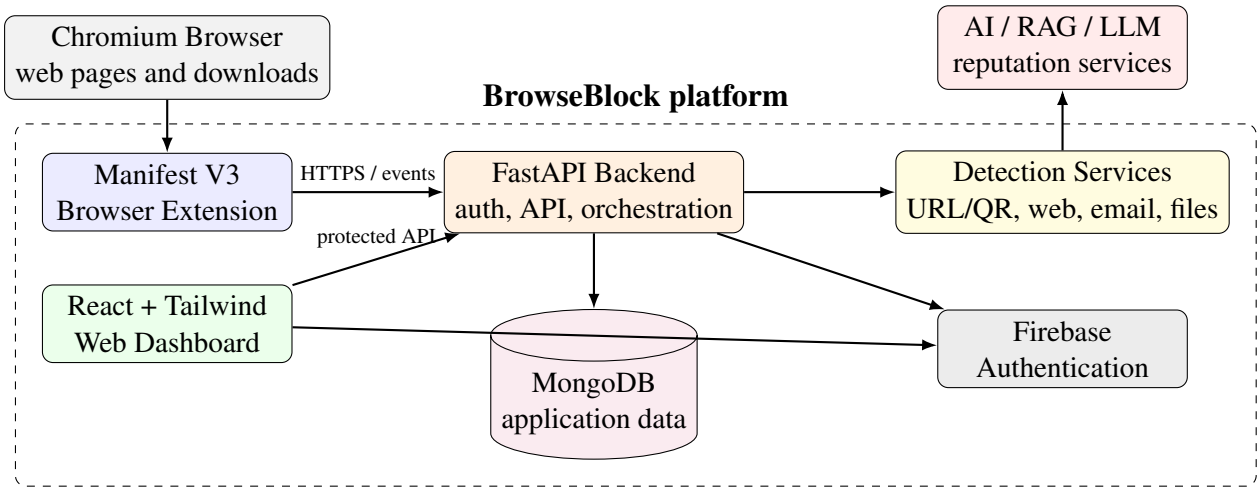
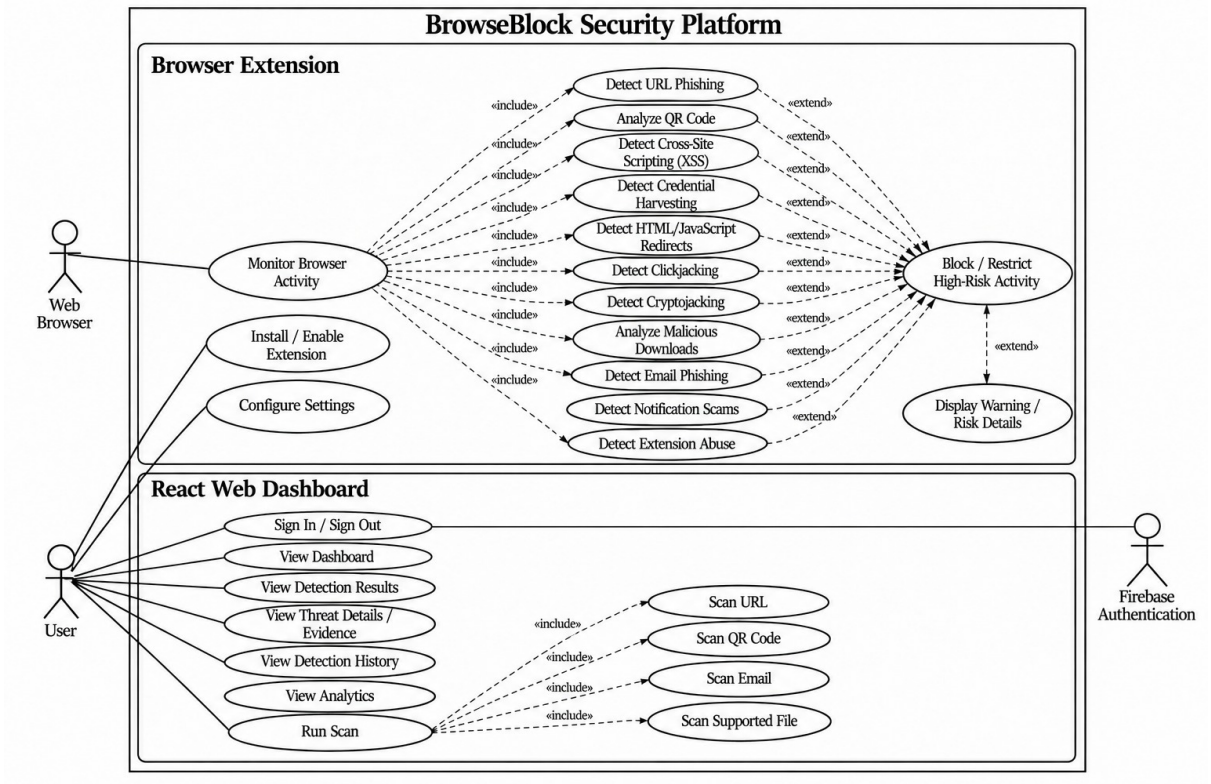


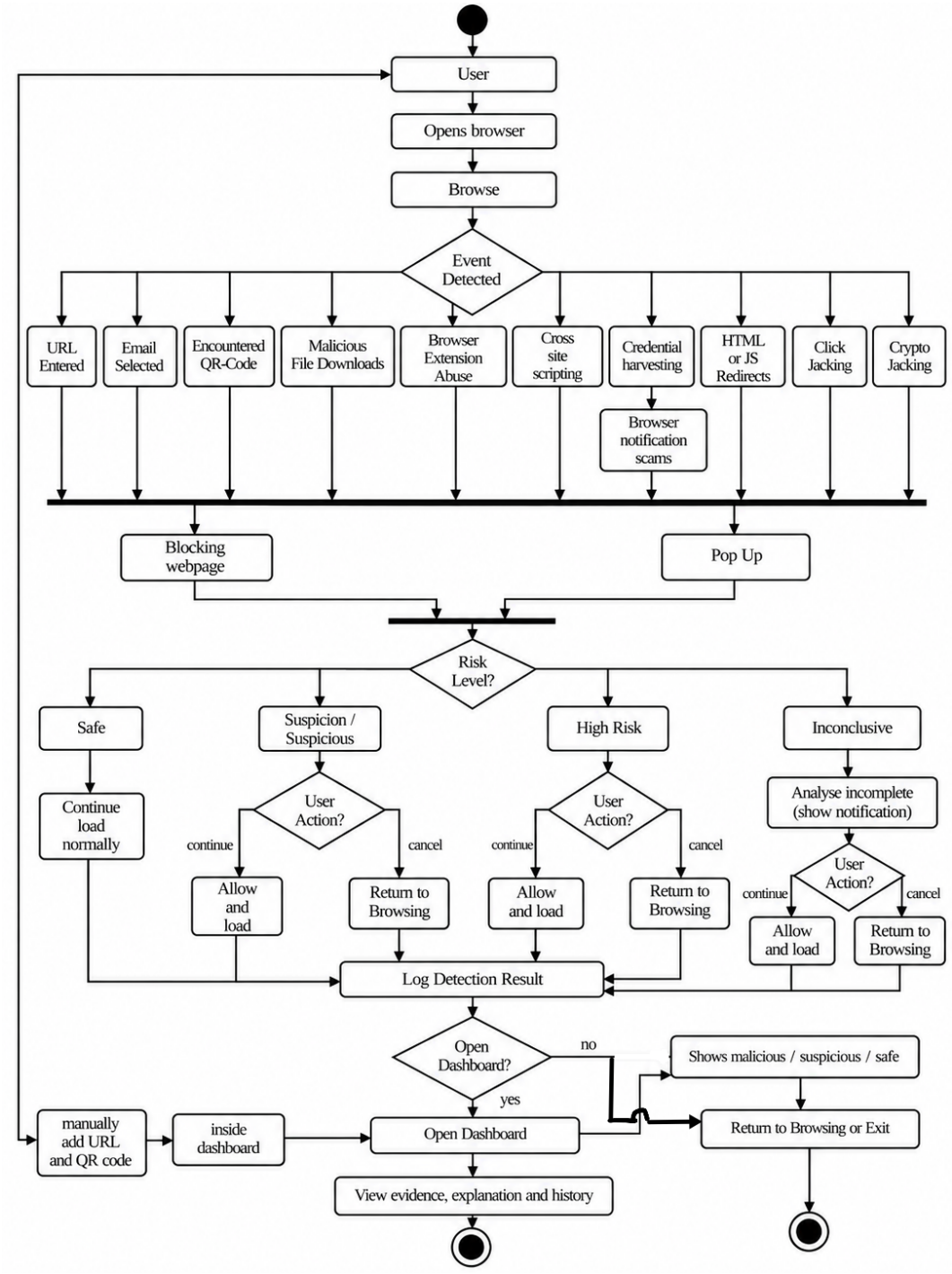
Figure 1: Logical architecture of the BrowseBlock platform

2.2 Use Case Diagram



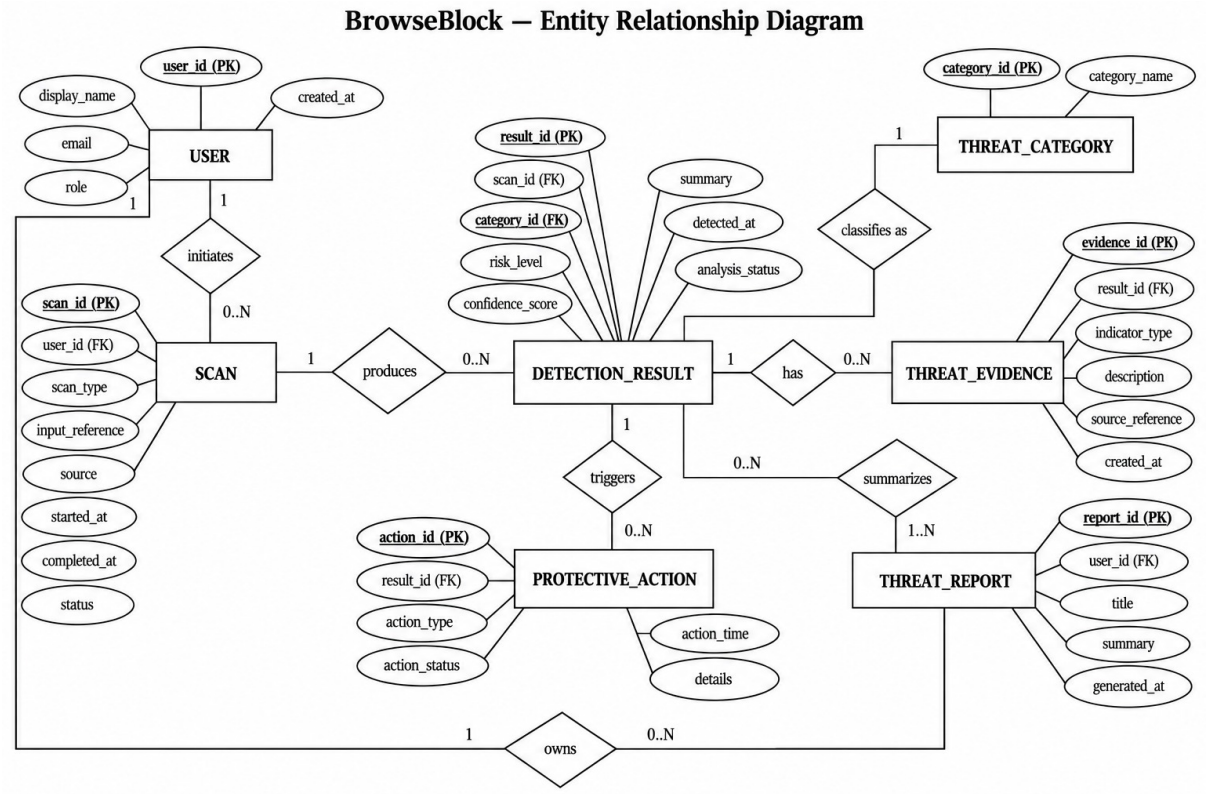
BrowseBlock use case diagram supplied with the SRS

2.3 Activity Diagram



BrowseBlock activity diagram supplied with the SRS

2.4 ER Diagram



BrowseBlock entity relationship diagram supplied with the SRS

3 Design Notes and Traceability

The design maps the SRS requirements to implementation boundaries as follows:

- URL and fake QR phishing requirements map to Module 3 and the URL/QR scan use cases.
- XSS, credential harvesting, redirects, clickjacking, cryptojacking, notification scams, and extension abuse map to Module 4.
- Malicious-download requirements map to Module 5, including static-only processing, resource limits, hash checks, and controlled quarantine.
- Email-phishing requirements map to Module 6 and its two-stage extension/ dashboard workflow.
- Authentication, privacy, HTTPS, token verification, MongoDB ownership, and role checks map to Modules 1, 8, and 10.
- Warning, blocking, cancellation, quarantine, incomplete analysis, and user-action logging map to Module 7 and the activity diagram.

The design intentionally preserves the limitations stated in the SRS. Browser permissions and API visibility can prevent complete observation; external services can be unavailable; model predictions can be uncertain; and a detected threat is not reported as blocked unless the configured protective action is confirmed.